Department of Information and Communication Technology
Faculty of Technology
University of Ruhuna

Mobile Application Development
ICT4153

Group Project
Group - 13

Digital Health Monitoring Platform
Project Report

Submitted By:
L.S.R. Vidanarachhchi

TG number	Name
TG-2021-1010	L.S.R. Vidanarachhchi
TG-2021-1038	B.D.D. Devindra
TG-2021-1056	M.A.S.V. Karunathilaka
TG-2021-1017	M.M.N.D.Seneviratne

Submitted to:
Mrs. W.A. Malsha Prabuddhi

30th of April 2026

Abstract

This report documents the design, architecture, and implementation of **Uplift**, a production-grade personal health monitoring application built with Flutter for Android. The application addresses fragmentation in the personal health technology space by consolidating step tracking, body measurement logging, goal management, and health education into a single, culturally relevant platform with bilingual support for English and Sinhala.

The core architectural decision is an **Offline-First, Dual-Storage** strategy: all data is written to a local SQLite database immediately, ensuring availability regardless of network state, while a background synchronisation engine mirrors data to Firebase Cloud Firestore for multi-device continuity. The technical stack comprises Flutter/Dart, Provider, Sqflite (schema v11), Firebase Auth, Cloud Firestore, fl_chart, syncfusion_flutter_gauges, and the Pedometer hardware API.

Table of Content

Abstract 1
1. Introduction & Project Context 3
1.1 Functional Requirements 3
3. System Architecture 4
3.1 Layered Repository Pattern 4
3.2 Offline-First Dual-Storage Strategy 5
4. Implementation Details 6
4.1 UI Architecture & Navigation 6
4.2 State Management & Business Logic 6
4.3 Database & Data Layer 6
4.4 API Integration & Device Features 9
5. Core Features & Optimizations 9
5.1 Bilingual Engine (English / Sinhala) 9
5.2 Dynamic Theming 9
5.3 Interactive Charts & Visualisations 9
5.4 Predictive Deadline Engine 9
5.5 Performance Optimization — Lazy Initialisation 9
5.6 Conditional Platform Database Initialisation 9
6. Challenges & Solutions 9
6.1 Dependency Version Conflict — intl Package 9
6.2 Gradle File Hash Lock on External Drives 9
6.3 Sinhala Font Rendering Overflow 9
6.4 Background Isolate User Identity 9
7. Conclusion & Future Enhancements 9
8. Individual Contribution Declaration 9
9. References 9
10. Appendices 9
Appendix A — Installation Guide 9
Appendix B — Dependencies Summary 9
Appendix C — Project File Structure 9

1. Introduction & Project Context

Existing health monitoring applications are largely designed for English-speaking, consistently connected users. This project addresses those gaps by providing a premium, offline-capable health monitor that generates predictive insights from user data rather than simply displaying raw numbers.

Uplift was developed as a group assignment simulating a commercial health-tech product for ICT4153 at the University of Ruhuna.

1.1 Functional Requirements

ID	Requirement
FR-01	Secure registration/login via Email/Password
FR-02	Google OAuth social authentication
FR-03	Persistent session management across restarts
FR-04	Automated step counting via device hardware sensor
FR-05	Manual activity and workout logging
FR-06	BMI auto-calculation from weight and height
FR-07	Extended body metrics (waist, hip, chest, body fat)
FR-08	Daily and cumulative goal tracking
FR-09	Goal-completion push notifications
FR-10	30-day interactive charts (activity, BMI, weight)
FR-11	Predictive goal deadline via linear regression
FR-12	Health Tips from external REST API with disk caching
FR-13	Favourites and Recently Viewed for Health Tips
FR-14	Profile photo via device Camera/Gallery
FR-15	Configurable reminders (banner and alarm modes)
FR-16	Light/Dark theme with database persistence
FR-17	Bilingual UI (English / Sinhala)

1.2 Non-Functional Requirements

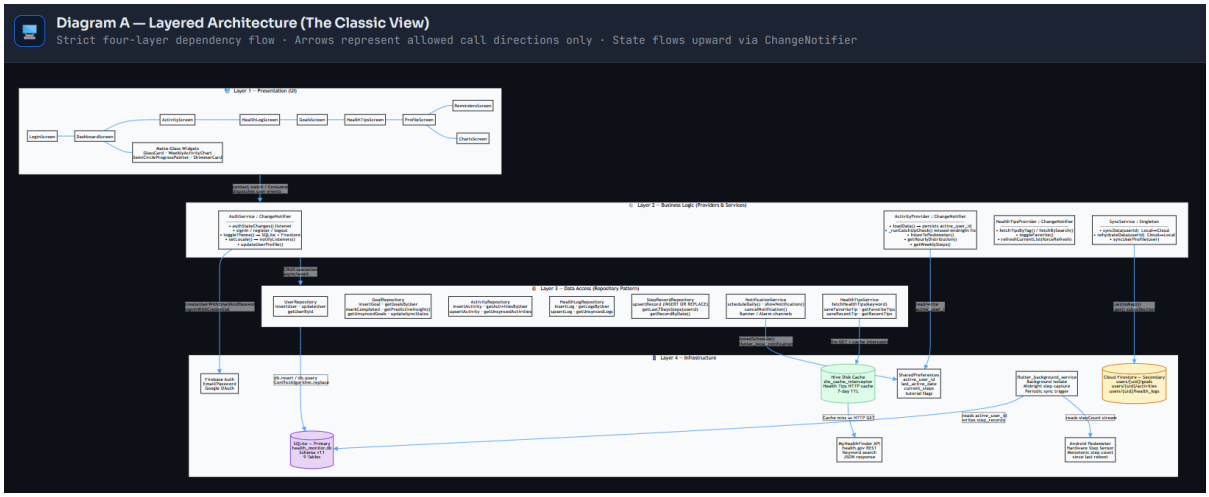
Category	Requirement
Data Integrity	Bidirectional sync (SQLite ↔ Firestore) with sync_status flags

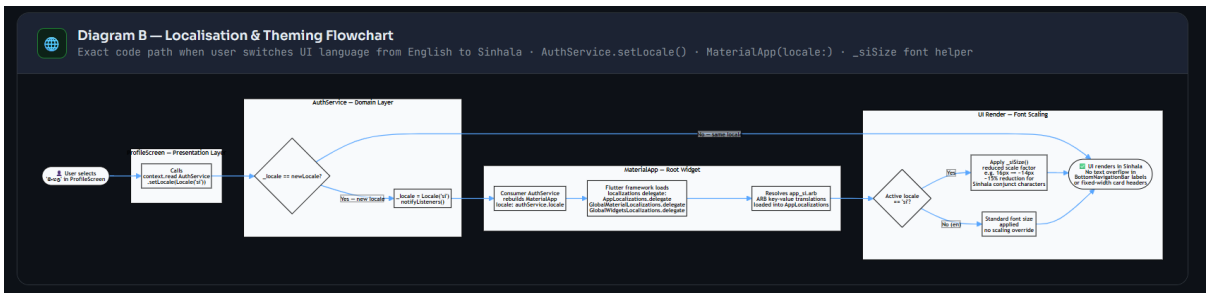
Availability	Full functionality without internet connectivity
Multi-Device	Data rehydrated from Firestore on login on any device
Performance	Lazy repository initialisation for zero-latency startup
Aesthetics	Dual-theme design system (Matte Alabaster / Solid Sapphire)
Usability	Locale-aware font scaling for Sinhala script
Background	Step tracking and sync continue while app is backgrounded

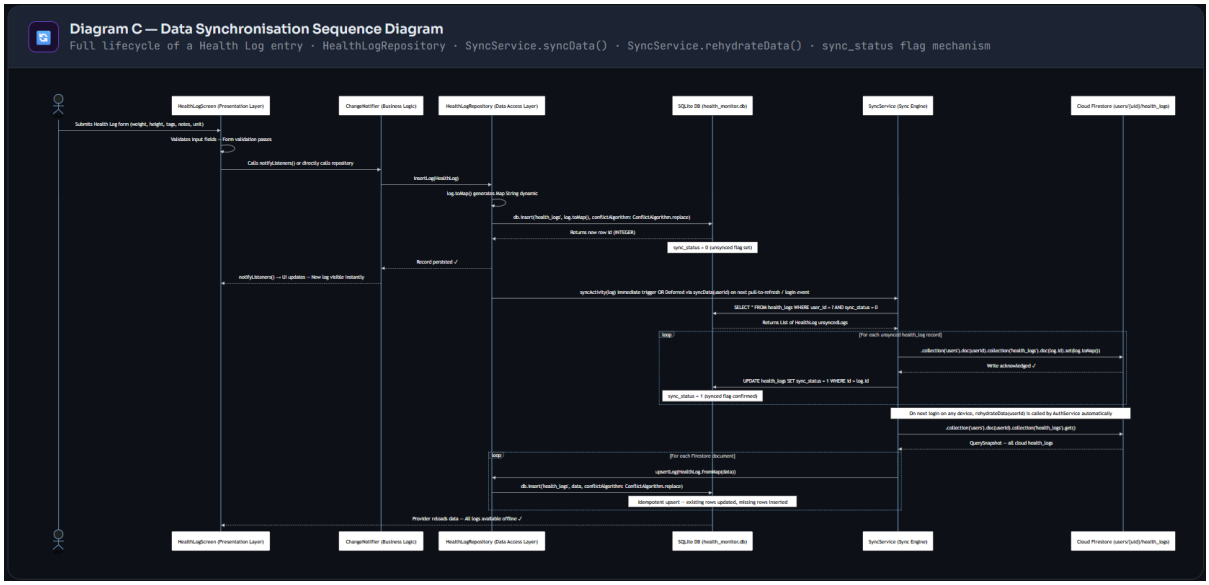
3. System Architecture

3.1 Layered Repository Pattern

The application enforces a strict three-layer architecture with a single-direction dependency flow. No layer communicates with a layer above it.







Presentation Layer — All visual components in lib/screens/ and lib/widgets/. Widgets consume Provider state and dispatch user events. They contain no database or network logic.

Domain Layer — AuthService, ActivityProvider, and HealthTipsProvider implement ChangeNotifier. This layer owns all business logic: authentication lifecycle, step delta calculation, predictive insight generation, and cache invalidation.

Data Layer — Repository classes (GoalRepository, ActivityRepository, HealthLogRepository, StepRecordRepository) provide an abstract CRUD interface. SyncService bridges SQLite and Firestore. DatabaseHelper owns the physical schema and all migration logic.

3.2 Offline-First Dual-Storage Strategy

Write Path (Local → Cloud): Every insert operation targets SQLite first with sync_status = 0. SyncService is called asynchronously — if online, the record is pushed to Firestore and sync_status is set to 1; if offline, the flag remains 0 and the record is picked up during the next syncData() cycle.

Read Path (Cloud → Local): On every login, AuthService calls SyncService.rehydrateData(userId), which fetches all Firestore subcollections and upserts them into SQLite using ConflictAlgorithm.replace. A fresh device instantly receives the user's complete history.

Zero-Configuration Scaling: Each model's .toMap() method is the shared translation layer consumed by both db.insert() (SQLite) and .set() (Firestore). Adding a new model field automatically extends both databases with zero additional configuration code.

4. Implementation Details

4.1 UI Architecture & Navigation

IndexedStack Navigation: The main scaffold uses IndexedStack with BottomNavigationBar. Unlike PageView, IndexedStack keeps all child widgets alive simultaneously — switching tabs preserves scroll positions and loaded state, delivering a native app experience.

Route Guards: The main.dart root widget wraps the app in a Consumer<AuthService>. When AuthService.isLoggedIn is false, the LoginScreen is rendered; when true, the MainScaffold is shown. Because AuthService calls notifyListeners() on every authStateChanges() event, this guard is fully reactive — logout from any screen immediately redirects the user.

Design System: The UI follows the "Matte Alabaster & Solid Sapphire" dual-theme documented in DESIGN.md. All colour values are defined in a central AppTheme class. Direct use of Colors.* Flutter primitives is prohibited — all colour references flow through AppTheme. Primary accent: Scooter (#2F9D94). Dark background: Sapphire (#0F172A).

4.2 State Management & Business Logic

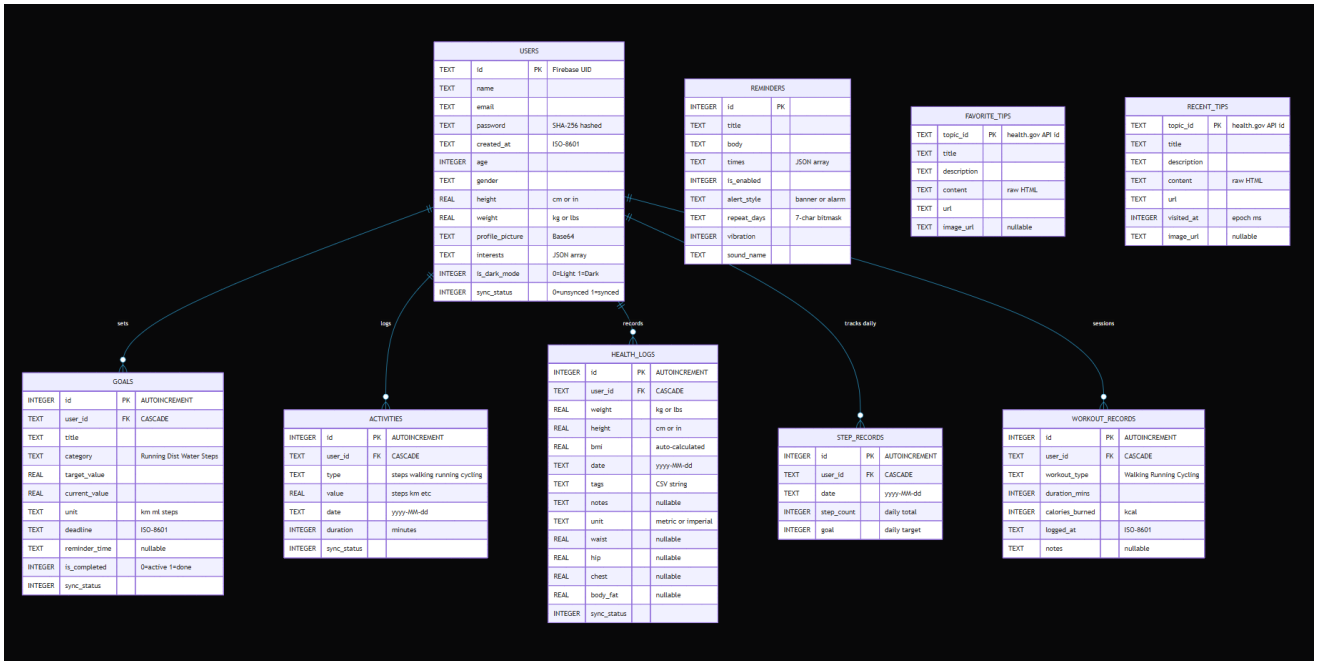
All state is managed via **Provider** with ChangeNotifier. A MultiProvider at the widget tree root registers all providers for dependency injection. The strict rule is: widgets render, providers think.

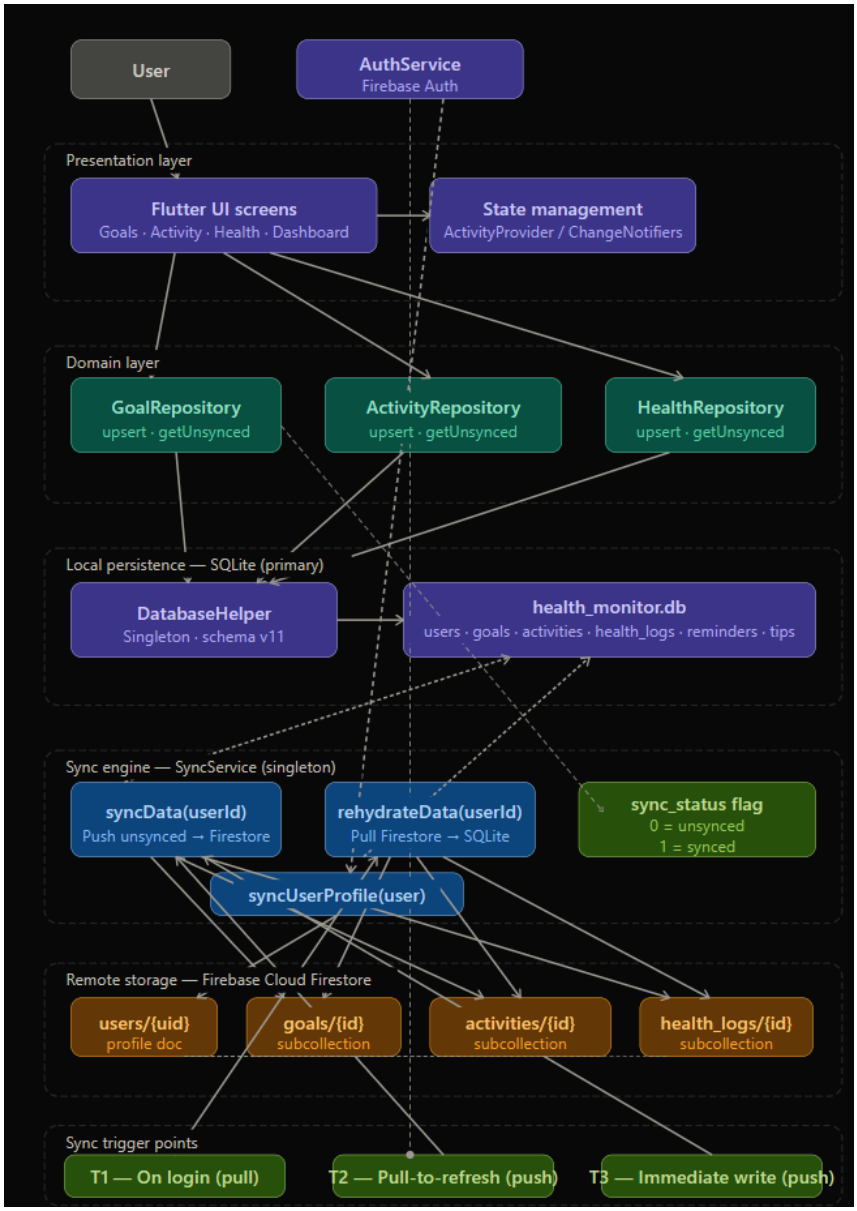
ActivityProvider Catch-Up Mechanism: On every app launch, _runCatchUpCheck() reads last_active_date from SharedPreferences. If it predates today, the cached current_steps value is inserted as a historical record for the missed date — ensuring no day is lost even if the background service failed due to a device power-off.

Background Isolate User ID: ActivityProvider.loadData() writes the authenticated UID to SharedPreferences under active_user_id. The BackgroundStepService runs in a separate isolate where Firebase Auth is inaccessible; it reads this key to identify the current user for step_records writes.

4.3 Database & Data Layer

The SQLite database is at **schema version 11**, upgraded incrementally via DatabaseHelper._onUpgrade().


USERS
TEXT id PK Firebase UID
TEXT name
TEXT email
TEXT password SHA:256 hashed
TEXT created_at ISO-8601
INTEGER age
TEXT gender
REAL height cm or in
REAL weight kg or lbs
TEXT profile_picture Base64
TEXT interests JSON array
INTEGER is_dark_mode 0=Light 1=Dark
INTEGER sync_status 0=unsynced 1=synced
REMINDERS
INTEGER id PK
TEXT title
TEXT body
TEXT times JSON array
INTEGER is_enabled
TEXT alert_style banner or alarm
TEXT repeat_days 7-char bitmask
INTEGER vibration
TEXT sound_name
FAVORITE_TIPS
TEXT topic_id PK health.gov API id
TEXT title
TEXT description
TEXT content raw HTML
TEXT url
TEXT image_url nullable
RECENT_TIPS
TEXT topic_id PK health.gov API id
TEXT title
TEXT description
TEXT content raw HTML
TEXT url
INTEGER visited_at epoch ms
TEXT image_url nullable
sets
logs
records
tracks daily
sessions
GOALS
INTEGER id PK AUTOINCREMENT
TEXT user_id FK CASCADE
TEXT title
TEXT category Running Dist Water Steps
REAL target_value
REAL current_value
TEXT unit km ml steps
TEXT deadline ISO-8601
TEXT reminder_time nullable
INTEGER is_completed 0=active 1=done
INTEGER sync_status
ACTIVITIES
INTEGER id PK AUTOINCREMENT
TEXT user_id FK CASCADE
TEXT type steps walking running cycling
REAL value steps km etc
TEXT date yyyy-MM-dd
INTEGER duration minutes
INTEGER sync_status
HEALTH_LOGS
INTEGER id PK AUTOINCREMENT
TEXT user_id FK CASCADE
REAL weight kg or lbs
REAL height cm or in
REAL bmi auto-calculated
TEXT date yyyy-MM-dd
TEXT tags CSV string
TEXT notes nullable
TEXT unit metric or imperial
REAL waist nullable
REAL hip nullable
REAL chest nullable
REAL body_fat nullable
INTEGER sync_status
STEP_RECORDS
INTEGER id PK AUTOINCREMENT
TEXT user_id FK CASCADE
TEXT date yyyy-MM-dd
INTEGER step_count daily total
INTEGER goal daily target
WORKOUT_RECORDS
INTEGER id PK AUTOINCREMENT
TEXT user_id FK CASCADE
TEXT workout_type Walking Running Cycling
INTEGER duration_mins
INTEGER calories_burned kcal
TEXT logged_at ISO-8601
TEXT notes nullable



Key Schema Decisions:

- users.id uses Firebase UID (TEXT PRIMARY KEY) as the unifying identity across local and cloud stores.
- users.profile_picture stores images as Base64 TEXT — no file-system permissions required.
- users.interests stores a JSON-encoded array for Health Tips personalisation.
- users.is_dark_mode persists theme preference as INTEGER 0/1, loaded at startup before the first frame renders.
- goals, activities, health_logs all carry sync_status INTEGER DEFAULT 0 — the flag that drives the sync engine.
- reminders is intentionally not user-scoped (no user_id FK) and not cloud-synced — it holds device-local notification schedules.

- step_records stores one row per user per day; background writes use ConflictAlgorithm.replace for idempotency.

Predictive Insights (GoalRepository): getPredictiveInsight() applies a linear regression to createdAt and currentValue to project the estimated completion date. For daily goals, it queries activities for today's actual sum rather than reading the static currentValue, preventing midnight carry-over inaccuracies. Output is a natural-language string passed to the UI.

Auto-Merge Architecture: When a user manually updates goal progress, GoalRepository calculates the delta and inserts a timestamped Activity record for the current date. This anchors all progress to its correct date in the activities table, producing accurate 30-day trend charts.

4.4 API Integration & Device Features

Firebase Authentication: AuthService listens to _auth.authStateChanges(). On sign-in, it calls _syncLocalUser(), which checks whether the user profile exists locally or in Firestore, then inserts or restores accordingly. Both Email/Password and Google OAuth flows are supported, with explicit error code handling for email-already-in-use, weak-password, and invalid-credential.

Health Tips REST API: The HealthTipsService integrates with the MyHealthFinder API (health.gov) using Dio with a DioCacheInterceptor backed by a HiveCacheStore. The caching policy is **Instant-First**: the UI renders from disk cache immediately; fresh data is only fetched on explicit pull-to-refresh. HTML content is rendered natively using flutter_widget_from_html.

Pedometer / Background Step Tracking: PedometerService interfaces with the Android hardware step sensor. Because the sensor returns a monotonically increasing total-since-reboot value, a _stepOffset is maintained in SharedPreferences to compute the accurate daily delta. BackgroundStepService runs via flutter_background_service, persisting daily totals to step_records with INSERT OR REPLACE.

Image Picker: Profile photos are captured via image_picker, encoded to Base64, stored in users.profile_picture, and decoded by the UI using MemoryImage(base64Decode(...)) — requiring no additional storage permissions.

Notification System: NotificationService implements two channels:

- **Banner** (health_reminders_banner): High-priority, auto-dismissing standard reminders.
- **Alarm** (health_reminders_alarm_v5): fullScreenIntent, FLAG_INSISTENT looping sound, custom vibration pattern, "Stop" action button, 120-second auto-timeout.

5. Core Features & Optimizations

5.1 Bilingual Engine (English / Sinhala)

The application implements Flutter's standard flutter_localizations and intl package infrastructure for internationalisation. Locale state is held in AuthService._locale as a Locale object and exposed via the locale getter. The setLocale(Locale) method calls notifyListeners(), which propagates the change to the root MaterialApp's locale property — triggering a full UI re-render in the selected language without requiring an app restart.

A key engineering challenge with Sinhala localisation is script rendering. The Sinhala Unicode block contains complex conjunct characters and ligatures that render wider than their Latin equivalents at the same fontSize. A global font-scaling helper function, referred to as _siSize(), was implemented to detect the active locale and apply a reduced font scale factor specifically for Sinhala text. This prevents text overflow in fixed-width UI components such as BottomNavigationBar labels and card headers — elements where layout constraints cannot accommodate variable-width script rendering.

5.2 Dynamic Theming

Theme state is managed by AuthService. The isDarkMode getter reads _currentLocalUser?.isDarkMode, falling back to a _forceDark boolean for unauthenticated sessions. When the user taps the theme toggle, toggleTheme() creates a new User model via copyWith(isDarkMode: !current) and calls updateUserProfile() — which writes the change to both SQLite (users.is_dark_mode) and Firestore via SyncService.syncUserProfile(). On the next app launch, _loadTheme() reads the most recently created user row from SQLite before the first frame renders, ensuring the correct theme appears with zero flash.

5.3 Interactive Charts & Visualisations

The ChartsScreen provides three distinct analytical tabs:

Activity Timeline: A separate 30-day bar chart is generated for every unique activity type the user has logged. Each chart reads from the activities table filtered by type over a 30-day trailing window.

Goal Insights — Cumulative Goals: A horizontal bar chart renders all cumulative goals together, showing progress percentage from 0% to 100% with a dashed target line overlaid at the targetValue Y-axis position.

Goal Insights — Daily Goals: Each daily goal receives its own 30-day line chart, showing daily fluctuations. Because daily goals reset at midnight, this visualisation correctly reads the

activities table by date rather than the goal's currentValue — producing historically accurate trend lines.

Weekly Activity Chart (WeeklyActivityChart): This fl_chart bar chart in the Activity section displays the last 7 days of step counts. A pre-pass computes the maximum value across all 7 bars before rendering begins, so the "best day" bar is correctly identified and highlighted with a gradient fill. Today's bar is labelled in bold. Interactive tap-to-tooltip behaviour displays the exact step count for any bar on touch.

BMI & Health Logs: Weight, height, and BMI are visualised as a correlated multi-line chart in the Health Log section, allowing users to observe the relationship between body composition changes over time.

5.4 Predictive Deadline Engine

Rather than displaying static progress percentages, the application generates forward-looking intelligence from the user's historical data. GoalRepository.getPredictiveInsight() analyses the velocity of progress — how much the user is achieving per day — and projects whether they will reach their target by the set deadline. The output is a human-readable, encouraging sentence surfaced directly in the Goal Insights UI, implemented as a natural-language string from the data layer to the presentation layer without any business logic leaking into the widget.

5.5 Performance Optimization — Lazy Initialisation

To achieve a zero-latency startup experience, all Repository and Service objects are instantiated using **lazy getter** patterns (get _goalRepo => GoalRepository()). This means the SQLite database connection, the Firestore client, and the notification plugin are not initialised until the first operation that requires them — typically a few hundred milliseconds after the first screen has already rendered. This approach prevents the 1–2 second startup delay common in applications that eagerly initialise all dependencies in main().

5.6 Conditional Platform Database Initialisation

The DatabaseHelper uses Dart's conditional import mechanism (import 'database_helper_stub.dart' if (dart.library.html) 'database_helper_web.dart') to isolate platform-specific database factory code. On mobile, the standard sqflite file-based database is used. On web (for demonstration purposes), sqflite_common_ffi_web provides an in-memory equivalent. This allows the same codebase to compile and run on both platforms without conditional logic polluting the business layer.

6. Challenges & Solutions

6.1 Dependency Version Conflict — intl Package

During the integration of the flutter_localizations SDK package (required for Sinhala locale support), a version conflict emerged with the intl package. The flutter_localizations SDK pinned intl at >=0.18.0 <0.19.0, while the syncfusion_flutter_gauges package and other dependencies required a more recent version. This created a pubspec resolution deadlock.

Solution: The conflict was resolved by running flutter pub upgrade --major-versions to force resolution to the latest compatible major version (intl: ^0.20.2), and by adding a dependency_overrides entry for vector_math: 2.3.0 in pubspec.yaml to resolve a secondary conflict introduced by flutter_widget_from_html pulling the package to an incompatible lower version.

6.2 Gradle File Hash Lock on External Drives

During development, the project was stored on an external SSD. Gradle's file hash cache uses file-system locking, which caused a Timeout waiting to lock file hash cache error when the project was opened after a prior crash — the lock file remained held by a stale PID referencing the previous process that had not cleanly released it.

Solution: The stale .lock file at android/.gradle/8.10.2/fileHashes/fileHashes.lock was manually deleted to release the lock. To prevent recurrence, kotlin.incremental=false was added to gradle.properties, which disables Kotlin's incremental compilation caching — the primary source of lock contention when the project directory and the Gradle/Pub cache reside on different physical drives.

6.3 Sinhala Font Rendering Overflow

When the UI locale was switched to Sinhala, several fixed-layout components — specifically the BottomNavigationBar label row and certain card headers — experienced text overflow. This occurred because Sinhala conjunct characters have a significantly larger rendered pixel width than the equivalent Latin character count would suggest.

Solution: A font-scaling utility function (_siSize) was implemented and applied globally to all text styles in the design system. The function detects the active locale from AuthService and returns a reduced scale factor for Sinhala text, preventing overflow without requiring per-widget layout adjustments.

6.4 Background Isolate User Identity

The flutter_background_service package runs the step-tracking task in a separate Dart isolate. Firebase Auth's currentUser getter is not accessible from a background isolate, making it impossible to determine which user's step_records row to write.

Solution: ActivityProvider.loadData() explicitly writes the authenticated user's UID to SharedPreferences under the key active_user_id. The background service reads this key during its task execution, removing any dependency on Firebase Auth from the background isolate.

7. Conclusion & Future Enhancements

Uplift successfully delivers a production-ready health monitoring platform that satisfies all seven advanced technical requirements of the ICT4153 module specification. The application demonstrates a robust, offline-first clean architecture that handles the full data lifecycle: from hardware sensor input through structured local persistence, intelligent background synchronisation, and rich interactive visualisation.

The project's most significant technical achievement is the **Offline-First Dual-Storage Sync Engine** — an architecture where SQLite is the primary source of truth and Firebase is an eventually consistent cloud replica, unified through a shared model .toMap() translation layer. This design gives the application the performance and reliability of a local-first app with the data continuity of a cloud-native one.

The **Predictive Insights Engine** and **Auto-Merge Goal-Activity Architecture** represent a meaningful advancement beyond basic CRUD functionality, placing intelligence directly in the data layer while maintaining strict separation of concerns.

Proposed Future Enhancements:

Enhancement	Description
AI Dietary Advice	Integration with a language model API to generate personalised meal recommendations based on logged health metrics
Wearable Integration	Bluetooth Low Energy (BLE) pairing with Fitbit or Garmin devices for automatic heart rate and sleep stage data
Expanded Biometrics	Blood pressure and blood glucose logging modules
Social Challenges	Group goal challenges between connected users via Firestore real-time listeners
iOS Deployment	Adapting the pedometer and notification permission flows for the iOS platform

8. Individual Contribution Declaration

The following table maps each group member to their primary deliverable area and the specific features they implemented, in accordance with the equal role distribution defined in the ICT4153 project specification.

Member	Responsible Layer	Key Features Implemented
MASV Karunathilaka (Member 1)	**UI Architecture & Navigation** — Layouts, responsiveness, routing	Health Log System UI; Application navigation framework (IndexedStack, BottomNavigationBar, route guards); Screen layouts and component architecture
MMND Senivirathne (Member 2)	**API Integration & Device Features** — Networking, async handling, plugin integration	Activity Tracking module; Real-time step counter (PedometerService, BackgroundStepService); ActivityProvider state management; Weekly and hourly chart visualisations
LSR Vidanaarachchi (Member 3)	**Database & Data Layer** — SQLite schema design, repository pattern	Firebase Authentication and Google OAuth; Profile management; Goal system; SQLite schema (v11); Repository pattern implementation; Predictive Insights engine; Auto-Merge architecture; Chart data layer; Dynamic theming; UI design contributions
BDD Devendra (Member 4)	**State Management & Business Logic** — Provider implementation, validation, logic separation	Localisation engine (English/Sinhala); Health Tips API integration (HealthTipsService, HealthTipsProvider); Disk caching strategy; Goal Reminder & Notification section (NotificationService); Bilingual font scaling

9. References

1. Flutter Team. *Flutter Documentation*. Google LLC. Retrieved April 2026. https://flutter.dev/docs

2. Firebase Team. *Firebase Documentation*. Google LLC. Retrieved April 2026. https://firebase.google.com/docs
3. Dart Team. *Dart Language Documentation*. Google LLC. Retrieved April 2026. https://dart.dev/guides
4. U.S. Department of Health & Human Services. *MyHealthFinder API v3*. health.gov. Retrieved April 2026. https://health.gov/our-work/national-health-initiatives/health-literacy/consumer-health-content/free-web-content/apis-developers
5. provider package — Remi Rousselet. https://pub.dev/packages/provider
6. sqflite package — Alexandre Roux. https://pub.dev/packages/sqflite
7. fl_chart package — Iman Khoshabi. https://pub.dev/packages/fl_chart
8. syncfusion_flutter_gauges — Syncfusion Inc. https://pub.dev/packages/syncfusion_flutter_gauges
9. pedometer package. https://pub.dev/packages/pedometer
10. flutter_background_service package. https://pub.dev/packages/flutter_background_service
11. flutter_local_notifications package — MichaelBui. https://pub.dev/packages/flutter_local_notifications
12. intl package — Dart Team. https://pub.dev/packages/intl
13. dio package — cfug. https://pub.dev/packages/dio
14. dio_cache_interceptor package. https://pub.dev/packages/dio_cache_interceptor
15. image_picker package — Flutter Team. https://pub.dev/packages/image_picker
16. permission_handler package — Baseflow. https://pub.dev/packages/permission_handler
17. google_sign_in package — Flutter Team. https://pub.dev/packages/google_sign_in

10. Appendices

Appendix A — Installation Guide

```
# 1. Clone the repository
git clone https://github.com/DuvinduDinethminDevendra/health_monitor-application.git
cd health_monitor-application
```

```
# 2. Install Flutter dependencies
flutter pub get

# 3. Run on a connected device or emulator
flutter run

# 4. For production build (Android APK)
flutter build apk --release
```

Minimum Requirements:

- Flutter SDK: >=3.3.0 <4.0.0
- Android: minSdkVersion 21 (Android 5.0+)
- A physical Android device is recommended for testing the Pedometer feature, as hardware step sensors are not emulated in virtual devices.

Appendix B — Dependencies Summary

Package	Version	Purpose
flutter_localizations	SDK	Bilingual UI (English/Sinhala)
provider	^6.1.2	Reactive state management
sqflite	^2.4.2	Local relational database
firebase_core	^4.7.0	Firebase SDK initialisation
firebase_auth	^6.4.0	Authentication
cloud_firestore	^6.3.0	Cloud data synchronisation
google_sign_in	^7.2.0	Google OAuth
fl_chart	^1.2.0	Interactive data charts
syncfusion_flutter_gauges	^33.2.4	Semi-circular progress gauge
pedometer	^4.0.2	Hardware step sensor access
flutter_background_service	^5.1.0	Background step tracking
flutter_local_notifications	^21.0.0	Push notifications & alarms
permission_handler	^12.0.1	Runtime permission requests
image_picker	^1.2.1	Camera / Gallery access
dio	^5.9.2	HTTP client
dio_cache_interceptor	^3.5.1	Intelligent response caching
dio_cache_interceptor_hive_store	^4.0.0	Disk-persistent cache store
shared_preferences	^2.5.2	Lightweight key-value storage
intl	^0.20.2	Internationalisation & date formatting
crypto	^3.0.3	SHA-256 password hashing
path_provider	^2.1.5	Device filesystem paths
flutter_animate	^4.5.2	UI micro-animations
shimmer	^3.0.0	Loading skeleton animations
cached_network_image	^3.4.1	Efficient network image caching

share_plus	^12.0.2	Native OS share sheet
url_launcher	^6.3.2	Open URLs in browser

Appendix C — Project File Structure

```
lib/
├── database/
│   ├── database_helper.dart       # SQLite singleton, schema v11, migrations
│   ├── database_helper_stub.dart   # Mobile factory
│   └── database_helper_web.dart    # Web factory (conditional import)
├── models/
│   ├── user.dart                  # User entity + toMap/fromMap
│   ├── goal.dart                  # Goal entity + baseType getter
│   ├── activity.dart              # Activity entity
│   ├── health_log.dart            # HealthLog entity + BMI calculation
│   ├── reminder.dart              # Reminder entity + AlertStyle enum
│   ├── step_record.dart           # Daily step record entity
│   └── workout_record.dart        # Workout session entity
├── repositories/
│   ├── user_repository.dart
│   ├── goal_repository.dart       # + getPredictiveInsight()
│   ├── activity_repository.dart
│   ├── health_log_repository.dart
│   └── step_record_repository.dart # + getLast7DaysSteps()
├── services/
│   ├── auth_service.dart          # Firebase Auth + ChangeNotifier
│   ├── sync_service.dart          # Bidirectional SQLite ↔ Firestore sync
│   ├── notification_service.dart   # Banner + Alarm channels
│   ├── pedometer_service.dart      # Hardware step sensor interface
│   ├── background_step_service.dart # Background isolate task
│   └── health_tips_service.dart    # Dio + cache + SQLite tips persistence
├── providers/
│   ├── activity_provider.dart      # Step state + catch-up mechanism
│   └── health_tips_provider.dart   # Tips state + search/tag/favourite logic
├── screens/                       # All application screens
├── widgets/                       # Reusable UI components
│   └── activity/
│       └── weekly_activity_chart.dart  # fl_chart bar chart with gradient
└── main.dart                      # App entry + MultiProvider + route guard
```